

Django vs FastAPI

A Practical Comparison for Backend Decision-Making

Kanishk

June 2026

Why This Document Exists

Django and FastAPI solve overlapping problems but come from different philosophies. Django is a *batteries-included* web framework built around the MVT pattern, an ORM, and an admin interface that ships for free. FastAPI is a *lightweight, async-first* framework built around type hints, automatic API documentation, and raw speed for I/O-bound workloads. Neither is strictly “better.” The right choice depends on what you are building. This document lays out the differences concretely, with code, so the trade-offs are visible rather than abstract.

1 Core Philosophy

Django: Convention over Configuration

Django ships with an ORM, an admin panel, a templating engine, a forms library, authentication, and a migration system — all built in and expected to be used together. You write less glue code because Django has already decided how the pieces fit.

FastAPI: Minimal Core, Explicit Choices

FastAPI gives you routing, request validation (via Pydantic), and automatic OpenAPI documentation. Everything else — the ORM, the admin UI, authentication — is a separate library you choose and wire in yourself (e.g. SQLAlchemy, Tortoise ORM, SQLModel).

2 Side-by-Side Comparison

Aspect	Django	FastAPI
Primary use case	Full websites: server-rendered pages, admin-heavy apps, CMS-style products	APIs and microservices, especially I/O-bound or ML-serving backends

Aspect	Django	FastAPI
Request model	Synchronous by default (async views supported since 3.1, but the ecosystem is still largely sync)	Async-first, built on Starlette and asyncio from the ground up
ORM	Built in (Django ORM)	None included; pair with SQLAlchemy, Tortoise ORM, or SQLModel
Admin interface	Built in, auto-generated from models	None; would need to build your own or use a third-party package
Data validation	Django Forms / Model validation	Pydantic models — validation comes from Python type hints directly
API docs	Not automatic (use Django REST Framework + drf-yasg/spectacular for OpenAPI)	Automatic interactive docs at <code>/docs</code> (Swagger UI) and <code>/redoc</code> , generated from your code with zero extra setup
Templating	Built-in Django Template Language (DTL) for server-rendered HTML	None built in; typically paired with a separate frontend (React, Vue) or Jinja2 if HTML is needed
Performance (raw I/O)	Slower under high concurrency unless paired with ASGI + async views	Significantly faster under concurrent I/O-bound load due to native async support
Learning curve	Steeper upfront (project structure, ORM, settings, admin) but very productive once learned	Shallower to start; mostly plain Python functions and type hints
Authentication	Built-in user model, sessions, permissions framework	Not included; implement manually or use libraries like fastapi-users
Ecosystem maturity	Released 2005; enormous third-party package ecosystem	Released 2018; smaller but rapidly growing, API-focused ecosystem

3 The Same Endpoint, Both Frameworks

To make the difference concrete, here is a single feature — “return a list of products with stock greater than zero” — written both ways.

3.1 Django (Class-Based View + DRF Serializer)

```
# models.py
class Product(models.Model):
    name = models.CharField(max_length=200)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)

# serializers.py (Django REST Framework)
```

```
class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price", "stock"]

# views.py
class InStockProductListView(generics.ListAPIView):
    serializer_class = ProductSerializer

    def get_queryset(self):
        return Product.objects.filter(stock__gt=0)

# urls.py
path("products/in-stock/", InStockProductListView.as_view())
```

Note: plain Django alone returns HTML, not JSON. To get a clean JSON API like this, you typically add **Django REST Framework** (DRF) on top — which is itself a strong sign of what Django's core priorities are.

3.2 FastAPI (Native Async + Pydantic)

```
# models.py (using SQLAlchemy, which merges Pydantic + SQLAlchemy)
class Product(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    price: float
    stock: int = 0

# main.py
@app.get("/products/in-stock", response_model=list[Product])
async def get_in_stock_products(session: Session = Depends(get_session)):
    statement = select(Product).where(Product.stock > 0)
    return session.exec(statement).all()
```

Note: FastAPI returns JSON by default. The function signature *is* the validation layer — no separate serializer class is needed because the type hints already describe the shape of the data.

4 When Django Wins

- You need an **admin panel** fast — e.g. internal tools, content-managed sites, anything where non-developers need to edit data.
- The product is a **traditional website** with server-rendered pages, not a pure API (blogs, e-commerce storefronts, dashboards).
- You want **batteries included**: auth, sessions, permissions, and ORM migrations working together out of the box.
- The team is larger or less experienced with async patterns — Django's conventions reduce the number of decisions every developer has to make independently.

5 When FastAPI Wins

- You are building a **pure API** or microservice, especially one consumed by a separate frontend or mobile app.
- The workload is **I/O-heavy and concurrent** — many simultaneous requests waiting on external calls (databases, third-party APIs, ML model inference).
- You want **automatic, accurate API documentation** with no extra configuration.
- You are integrating with **machine learning models**, where FastAPI has become close to a default choice for serving predictions.

6 A Note on Async

This is the single biggest architectural difference and worth understanding on its own terms, independent of either framework.

- **Synchronous** code (classic Django views) processes one request at a time per worker. If a view is waiting on a slow database query, that worker is blocked and cannot serve another request in the meantime. You scale by adding more worker processes.
- **Asynchronous** code (FastAPI, async Django views) can pause a request while it waits on I/O (a database call, an HTTP request to another service) and use that same worker to handle a different request in the meantime. This matters most when requests spend more time *waiting* than *computing*.

Django *can* do async now (since 3.1, with ASGI servers like Daphne or Uvicorn), but the ORM and most of the ecosystem remain primarily synchronous. FastAPI was async from its first release, so the entire ecosystem around it assumes it.

7 Practical Takeaway

Rule of Thumb

If the project needs an admin panel, server-rendered pages, or a large built-in feature set — reach for **Django**.

If the project is a clean API, especially one with heavy concurrent I/O or ML serving — reach for **FastAPI**.

Many real systems use **both**: Django for the admin-facing, content-managed side of a product, and a separate FastAPI service for a high-throughput API layer.